Массивы в C#

Массивы в C#

- упорядоченная совокупность элементов одного и того же типа, объединенных одним именем
- относятся к ссылочным типам данных
- реализованы как объекты
  - имя массива является ссылкой на область динамической памяти, в которой последовательно размещается набор элементов определенного типа
  - выделение памяти под элементы происходит на этапе инициализации массива
  - за освобождением памяти следит система сборки мусора: неиспользуемые массивы автоматически утилизируются

Оператор foreach

- применяется для перебора элементов в специальным образом организованной группе данных, в том числе в массиве
- Синтаксис:
- foreach (<тип> <имя> in <группа>) <тело цикла>
  - *имя* определяет локальную по отношению к циклу переменную, которая будет по очереди принимать все значения из указанной *группы*
  - *тип* соответствует базовому типу элементов *группы*
- Ограничение: с помощью foreach можно только просматривать значения элементов в группе данных, но нельзя их изменять

Одномерный массив

- нумерация элементов начинается с нуля
- реализуется как объект, поэтому его создание представляет собой двухступенчатый процесс:
  - объявляется ссылочная переменная на массив
  - выделяется память под требуемое количество элементов базового типа и ссылочной переменной присваивается адрес нулевого элемента в массиве


Способы описания одномерного массива
Без инициализации
Пример:
int [] MyArray1;
double [] MyArray2;
string [] MyArray3;
С инициализацией
Инициализация с использованием конструктора
Явная инициализация
Пример:
double[] MyArray1 = {1,2.4,1.7};
bool[] MyArray2 = {true, false};
char[] MyArray3 = {'a','e','i'};
Инициализация с использованием конструктора без указания значений элементов
Пример:
int [] MyArray1 = new int[10];
bool [] MyArray2 = new bool[5];
Инициализация с использованием конструктора с указанием значений элементов
Без указания размерности массива
Пример:
int[] MyArray = new int[]{1,2};
С указанием размерности массива
Пример:
int[] MyArray = new int[2]{5,2};

- Примеры 1-10

Массив как объект

- Массивы реализованы на основе базового класса Array, определенного в пространстве имен System

Свойства и методы класса Array

Элемент	*Вид*	*Описание*
Length	свойство	Количество элементов массива (по всем размерностям)
BinarySearch()	статический метод	Двоичный поиск в отсортированном массиве
Clear()	статический метод	Присваивание элементам массива значений по умолчанию
Copy()	статический метод	Копирование заданного диапазона элементов одного массива в другой
CopyTo()	экземплярный метод	Копирование всех элементов текущего одномерного массива в другой массив

Свойства и методы класса Array

Элемент	*Вид*	*Описание*
Find()	статический метод	Поиск первого элемента, удовлетворяющего заданному условию
FindAll()	статический метод	Возвращает одномерный массив элементов, удовлетворяющих заданному условию
FindIndex()	статический метод	Возвращает индекс первого вхождения элемента, удовлетворяющего заданному условию
FindLast()	статический метод	Возвращает последнее вхождение элемента, удовлетворяющего заданному условию
FindLastIndex()	статический метод	Возвращает индекс последнего вхождения элемента, удовлетворяющего заданному условию

Свойства и методы класса Array

Элемент	*Вид*	*Описание*
GetValue()	экземплярный метод	Получение значения элемента массива
IndexOf()	статический метод	Поиск первого вхождения элемента в одномерный массив
LastIndexOf()	статический метод	Поиск последнего вхождения элемента в одномерный массив
Rank()	свойство	Количество измерений массива
Reverse()	статический метод	Изменение порядка следования элементов на обратный
SetValue()	экземплярный метод	Установка значения элемента массива
Sort()	статический метод	Упорядочивание элементов одномерного массива

- Вызов статических методов происходит через обращение к имени класса
  - Array.Sort(myArray)
- Обращение к свойству или вызов экземплярного метода производится через обращение к экземпляру класса
  - myArray.Length или myArray.GetValue(i)

- Примеры 11-14

Массив как параметр

- Так как имя массива фактически является ссылкой, то он передается в метод по ссылке и, следовательно, все изменения элементов массива, являющегося формальным параметром, отразятся на элементах соответствующего массива, являющегося фактическим параметром
- Примеры: 15, 16

Массив параметров *params*

- позволяет передавать в метод переменное число параметров
- В списке параметров не может быть больше одного такого параметра, причем он всегда стоит последним (последнему формальному параметру, если он объявлен с ключевым словом *params*, ставятся в соответствие все оставшиеся фактические аргументы)
- Пример 17

Многомерные массивы

- Нумерация строк и столбцов начинается с нуля
- Синтаксис:

<ТипЭлементов>[,{ ,..,}] <ИмяМассива>{<Инициализация>};

Формы записи

- тип [,] ИмяМассива;
- тип [,] ИмяМассива={{элементы 1-ой строки}, … , {элементы n-ой строки}};
- тип [,] ИмяМассива = new тип [размер1, размер2];
- тип [,] ИмяМассива= new тип [,]{{элементы 1-ой строки}, … ,{элементы n-ой строки}};
- *Например*:
- int [,] a;
- int [,] a={{0, 1, 2}, {3, 4, 5}};
- int [,] a= new int [3, 4];
- int [,] a= new int [,]{{0, 1, 2}, {3, 4, 5}};
-

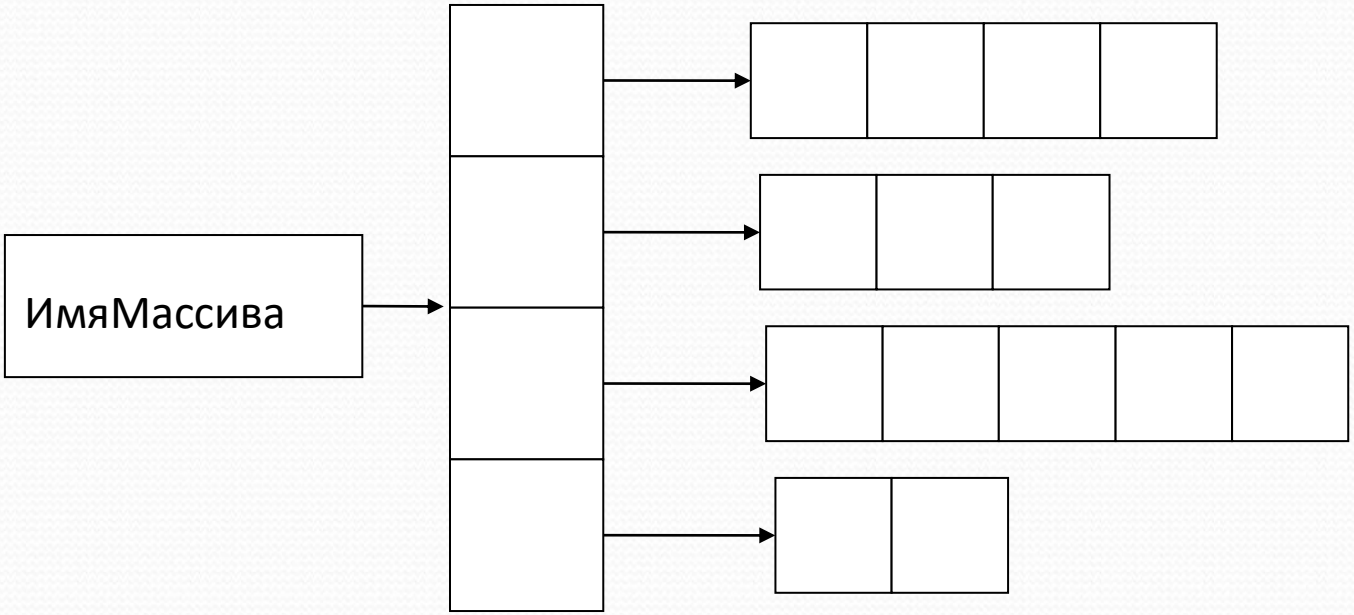
- Последний способ описания является избыточным
- При работе с многомерными массивами можно использовать приемы, которые мы рассмотрели для одномерных массивов
- При обращении к свойству Length для двумерного массива получим общее количество элементов в массиве
  - GetLength(0) – количество строк
  - GetLength(1) – количество столбцов

  Примеры 18-19

Ступенчатые массивы

- В отличие от прямоугольных массивов, в ступенчатых массивах количество элементов в разных строках может быть различным
- Синтаксис:

<ТипЭлементов>[ ][ ] <ИмяМассива> {<Инициализация>};


ИмяМассива

Пример

```
int[ ][ ] JaggedArray = new int[4][ ];
JaggedArray[0] = new int[4]; JaggedArray[1] = new int[3];
JaggedArray[2] = new int[5]; JaggedArray[3] = new int[2];
```

-

```
int[ ][ ] JaggedArray = { new int[4], new int[3],
                          new int[5], new int[2] };
```

- Пример 20